Statistical Computing

Optimization Algorithms

Spring 2026

Outline

Unconstrained Optimization

Definitions

Statements for **minimum** apply to **maximum** by flipping $\leq$ to $\geq$ (equivalently, $\max f = -\min(-f)$).

- **Unconstrained problem:** minimize (or maximize) $f(x)$ over $x \in \mathbb{R}^n$.
- $\bar{x} \in \mathbb{R}^n$ is a **global minimum** of f if $f(\bar{x}) \leq f(x)$ for all $x \in \mathbb{R}^n$.
- $\bar{x} \in \mathbb{R}^n$ is a **local minimum** of f if there exists $\varepsilon > 0$ such that $f(\bar{x}) \leq f(x)$ for all x with $\|x - \bar{x}\| < \varepsilon$; a **strict local minimum** if the inequality is strict for every such $x \neq \bar{x}$.
- If f is differentiable and $\nabla f(\bar{x}) = 0$, then $\bar{x}$ is a **critical point** of f.

Descent Direction

Definition

Suppose $f : \mathbb{R}^n \to \mathbb{R}$ is differentiable at $\bar{x}$. A vector $d \in \mathbb{R}^n$ is a **descent direction** of f at $\bar{x}$ if

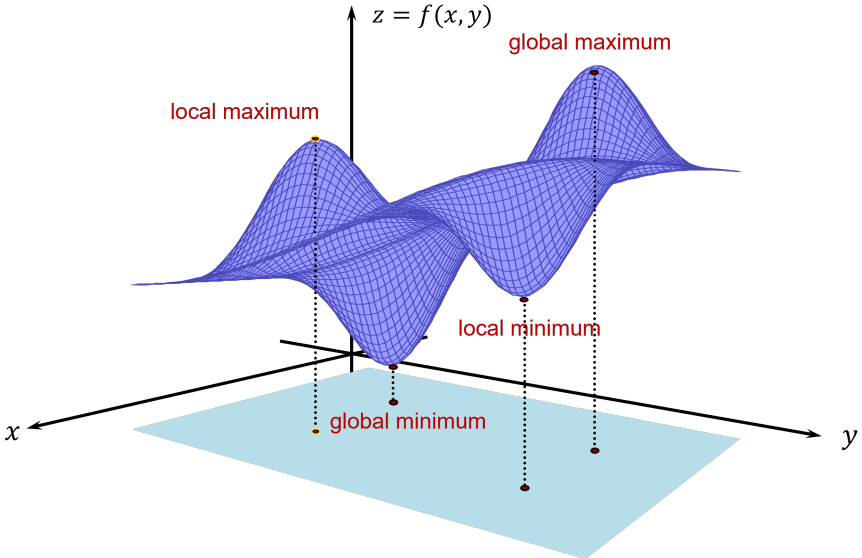
$$\nabla f(\bar{x})^\mathsf{T} d < 0.$$

Theorem (Descent Lemma)

If d is a descent direction of f at $\bar{x}$, then there exists $\delta > 0$ such that

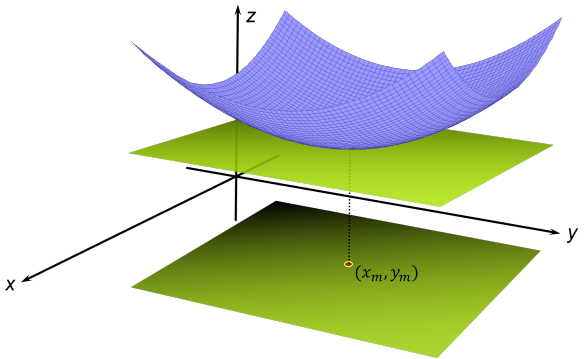
$$f(\bar{x} + \lambda d) < f(\bar{x}) \qquad \text{for every } \lambda \in (0, \delta).$$

Graphic Example



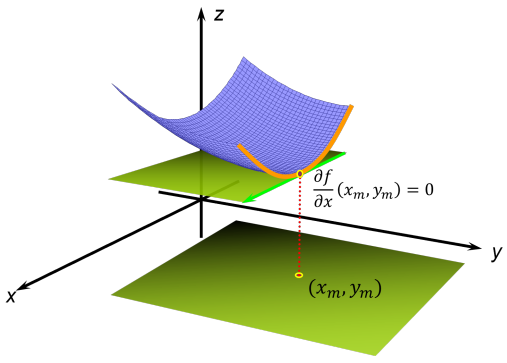
Local Minima

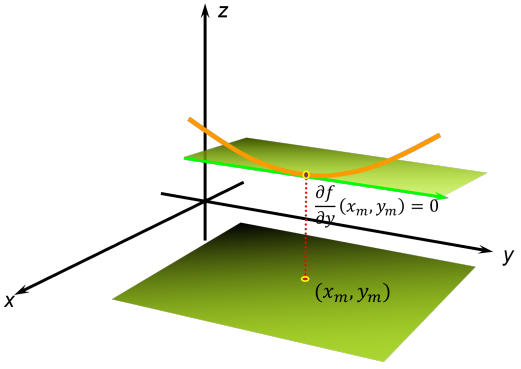
- At a minimum point of the graph of a function of two variables, such as point (x_m, y_m) below, the plane tangent to the graph of the function is horizontal (assuming the surface of the graph is smooth):



Local Minima: Zero Partial Derivatives

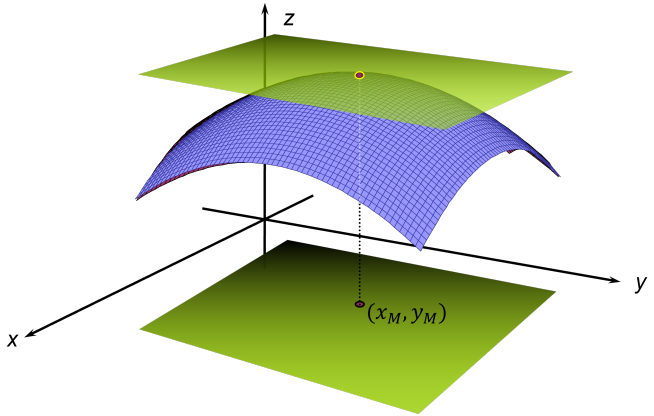
- At a minimum point, the graph of the function has a slope of zero along a direction parallel both to the x-axis and y-axis, respectively.





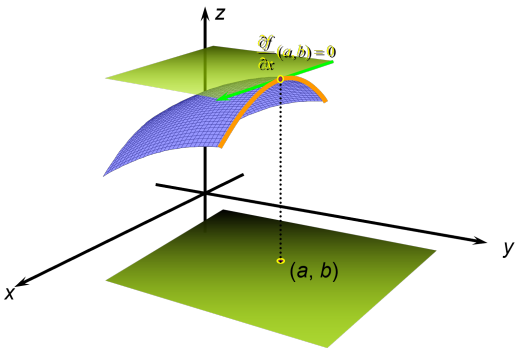
Local Maxima

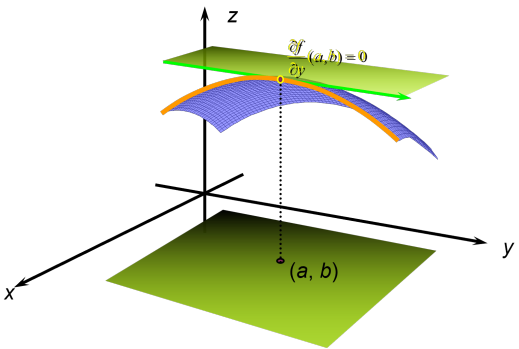
- At a maximum point of the graph of a function of two variables, such as point (a, b) below, the plane tangent to the graph of the function is horizontal (assuming the surface of the graph is smooth):



Local Maxima: Zero Partial Derivatives

- At a maximum point, the graph of the function has a slope of zero along a direction parallel both to the x-axis and y-axis, respectively.

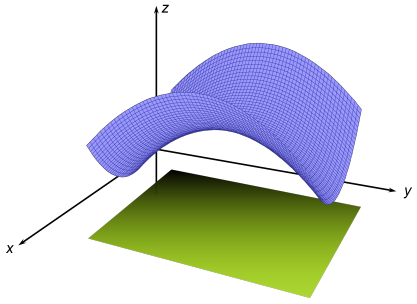




Saddle Point

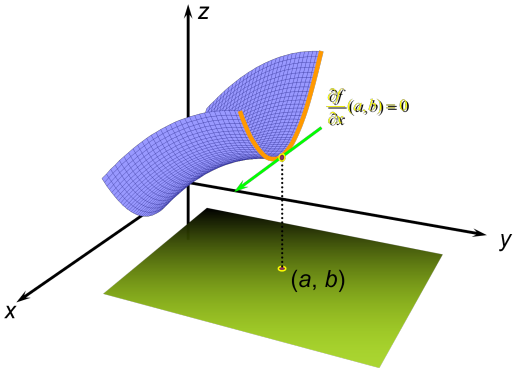
Definition

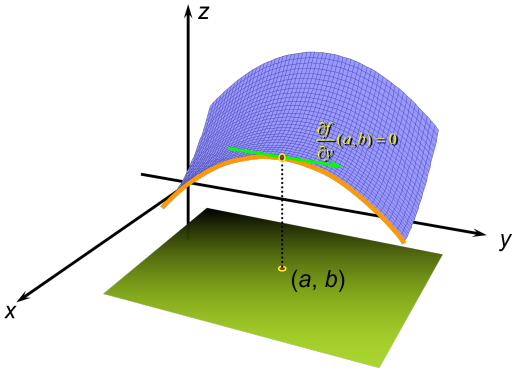
A critical point $\bar{x}$ of f is a **saddle point** if every neighborhood of $\bar{x}$ contains points x_1, x_2 with $f(x_1) < f(\bar{x}) < f(x_2)$ (i.e., neither a local min nor a local max).



Saddle Point: Min and Max Along Perpendicular Planes

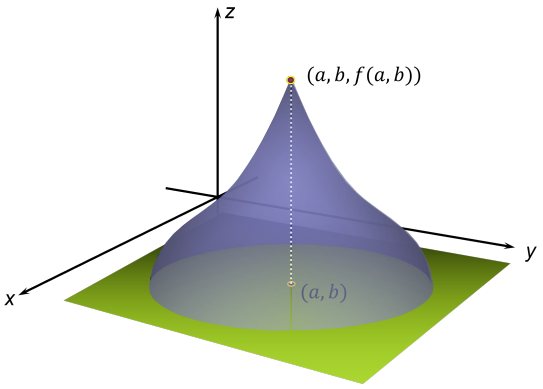
- In the case of a saddle point, the function is at a minimum along one vertical plane...
- ...but at a maximum along the perpendicular vertical plane.





Extrema When Partial Derivatives Are Not Defined

- A maximum (or minimum) may also occur when both partial derivatives are not defined, such as point (a, b) in the graph below:



First-Order Necessary Optimality Conditions

Suppose $f : \mathbb{R}^n \to \mathbb{R}$ is differentiable at $\bar{x}$. If $\bar{x}$ is a local minimum, then $\nabla f(\bar{x}) = 0$.

Proof

If $d := -\nabla f(\bar{x}) \neq 0$, then by the first-order approximation,

$$f(\bar{x} + \lambda d) \approx f(\bar{x}) + \lambda \nabla f(\bar{x})^{\mathsf{T}} d = f(\bar{x}) - \lambda \left\| \nabla f(\bar{x}) \right\|^2 < f(\bar{x}).$$

This contradicts $\bar{x}$ being a local minimum, so $\nabla f(\bar{x}) = 0$. □

Second-Order Optimality Conditions

Suppose $f : \mathbb{R}^n \to \mathbb{R}$ is twice differentiable at $\bar{x}$.

- **[Necessary]** If $\bar{x}$ is a local minimum, then $\nabla f(\bar{x}) = 0$ and $\nabla^2 f(\bar{x})$ is positive semidefinite.
- **[Sufficient]** If $\nabla f(\bar{x}) = 0$ and $\nabla^2 f(\bar{x})$ is positive definite, then $\bar{x}$ is a strict local minimum.

Determining Local Optima

1. Find the critical points of $f(x, y)$ by solving the simultaneous equations

$$f_x = 0, \qquad f_y = 0.$$

2. The **second derivative test**: Let $D(x, y) = f_{xx} f_{yy} - f_{xy}^2$.
3. Then at a critical point (a, b):
   - 3.1 $D(a, b) > 0$ and $f_{xx}(a, b) < 0 \Rightarrow$ **local maximum**.
   - 3.2 $D(a, b) > 0$ and $f_{xx}(a, b) > 0 \Rightarrow$ **local minimum**.
   - 3.3 $D(a, b) < 0 \Rightarrow$ **saddle point**.
   - 3.4 $D(a, b) = 0 \Rightarrow$ test is **inconclusive**.

Example

Problem. Find the local extrema of $f(x, y) = 4y^3 + x^2 - 12y^2 - 36y + 2$.

Solution.

- $f_x = 2x$ and $f_y = 12y^2 - 24y - 36 = 12(y + 1)(y - 3)$.
- Setting $f_x = 0$ and $f_y = 0$: $x = 0$ and $y = -1$ or $y = 3$.
- Two critical points: $(0, -1)$ and $(0, 3)$.
- $f_{xx} = 2$, $f_{yy} = 24(y - 1)$, $f_{xy} = 0$, so

$$D(x, y) = f_{xx} f_{yy} - f_{xy}^2 = 48(y - 1).$$

- At $(0, -1)$: $D(0, -1) = 48(-2) = -96 < 0 \Rightarrow$ **saddle point**.
  - Saddle point value: $f(0, -1) = 22$, so the saddle point is at $(0, -1, 22)$.

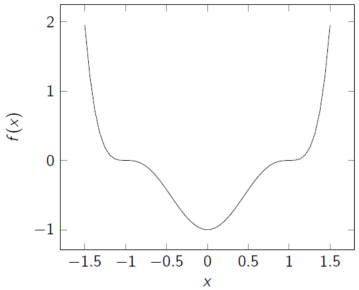
Example (continued)

Solution (continued).

- At $(0, 3)$: $D(0, 3) = 48(2) = 96 > 0$ and $f_{xx}(0, 3) = 2 > 0$.
- Therefore f has a **local minimum** at $(0, 3)$.
- The local minimum value is $f(0, 3) = -106$, so the local minimum is at $(0, 3, -106)$.

Unconstrained Example

- Example: $\min_x \; f(x) = (x^2 - 1)^3$.
- $f'(x) = 6x(x^2 - 1)^2 = 0$ for $x = 0, \pm 1$.
- $H(x) = f''(x) = 24x^2(x^2 - 1) + 6(x^2 - 1)^2$, so $H(0) = 6 > 0$ and $H(\pm 1) = 0$.
- Therefore $x = 0$ is a local minimum (in fact, the global minimum).



Unconstrained Example: Classifying $x = \pm 1$

- At $x = \pm 1$ the second-order test is **inconclusive** ($H = 0$): could be min, max, or saddle.
- Check the definition directly. For small $\epsilon > 0$,

$$f(1-\epsilon) < 0 = f(1) < f(1+\epsilon), \qquad f(-1-\epsilon) > 0 = f(-1) > f(-1+\epsilon),$$

  so every neighborhood of $x = \pm 1$ contains points with values both above and below $f(\pm 1)$.
- Hence $x = \pm 1$ are **saddle points**.

Example: Maximum Likelihood (Univariate Gaussian)

The univariate Gaussian distribution:

$$p(x \mid \mu, \sigma^2) = \frac{1}{(2\pi)^{1/2}\sigma} \exp\left(-\frac{1}{2\sigma^2}(x-\mu)^2\right),$$

where μ is the mean and $\sigma^2 = \mathbb{E}\left[(x-\mu)^2\right]$ is the variance.

$$\begin{aligned}
\mu_{\mathrm{ML}} &= \arg\max_{\mu}\ p(x_1, \ldots, x_n \mid \mu, \sigma^2) = \arg\max_{\mu}\ \prod_{i=1}^{n} p(x_i \mid \mu, \sigma^2) \quad \text{(i.i.d.)} \\
&= \arg\max_{\mu}\ \sum_{i=1}^{n} \ln p(x_i \mid \mu, \sigma^2) \quad \text{(monotonicity of log)} \\
&= \arg\min_{\mu}\ \sum_{i=1}^{n} (x_i - \mu)^2.
\end{aligned}$$

Example: Maximum Likelihood (Solving for MLE)

- For which $\boldsymbol{\theta} = (\mu, \sigma^2)$ are $x_1, \ldots, x_n$ most likely, given $x_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\mu, \sigma^2)$?

$$\mathrm{LL} \;:=\; \log p(x_1, \ldots, x_n \mid \mu, \sigma^2) = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2.$$

Setting partial derivatives to zero:

$$\frac{\partial \mathrm{LL}}{\partial \mu} = \frac{1}{\sigma^2}\sum_{i=1}^{n}(x_i - \mu) = 0 \quad \Longrightarrow \quad \mu_{\mathrm{ML}} = \frac{1}{n}\sum_{i=1}^{n} x_i,$$

$$\frac{\partial \mathrm{LL}}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4}\sum_{i=1}^{n}(x_i - \mu)^2 = 0 \quad \Longrightarrow \quad \sigma^2_{\mathrm{ML}} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu_{\mathrm{ML}})^2.$$

Optimization Algorithms

Line Search Method

- The **line search method** is an iterative approach in optimization algorithms that updates the solution at each step k as

$$x_{k+1} = x_k + \eta_k \, d_k.$$

- Where:
  - d_k: **search direction**.
  - η_k: **learning rate** or **step size**.
- The core challenge of line search methods is selecting a good direction d_k and an appropriate step size η_k.

Definition

Most line search algorithms require the search direction d_k to be a **descent direction**:

$$d_k^{\mathrm{T}} \nabla f_k < 0.$$

- The search direction can generally be defined in the form

$$d_k = -B_k^{-1} \nabla f_k,$$

where $B_k \in \mathbb{R}^{n \times n}$ is a symmetric and nonsingular matrix. The choice of B_k defines the specific method.

Choice of B_k

- **Gradient Descent:** $B_k = I$ (identity matrix).
- **Newton's Method:** $B_k = \nabla^2 f(x_k)$ (Hessian matrix).
- **Quasi-Newton Methods:** B_k is an approximation of the Hessian, typically updated via low-rank modifications.

If $B_k \succ 0$ (positive definite), then so is B_k^{-1}, hence

$$d_k^{\mathsf{T}} \nabla f_k \;=\; -\nabla f_k^{\mathsf{T}} B_k^{-1} \nabla f_k \;<\; 0.$$

Thus, d_k is guaranteed to be a descent direction.

Gradient Descent

The simplest choice $B_k = I$ gives **gradient (steepest) descent**:

$$x_{k+1} \;=\; x_k - \eta_k \, \nabla f_k .$$

Why $-\nabla f$? For a unit-norm direction d, the first-order approximation

$$f(x_k + \eta\, d) \;\approx\; f(x_k) + \eta\, \nabla f_k^{\mathsf{T}} d$$

decreases fastest when $\nabla f_k^{\mathsf{T}} d$ is most negative. By Cauchy–Schwarz,

$$\nabla f_k^{\mathsf{T}} d \;\geq\; -\,\|\nabla f_k\|\,,$$

with equality at $d = -\nabla f_k / \,\|\nabla f_k\|$. So $-\nabla f_k$ is the **direction of steepest descent**.

Gradient Descent: Convergence Behavior

- Convergence rate is **linear** (with a Wolfe line search).
- Speed depends strongly on the **condition number** of the Hessian:

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

- Large κ (ill-conditioned problem) $\Rightarrow$ very **slow progress**.
- This is the main motivation for Newton-type and quasi-Newton methods.

Learning Rate

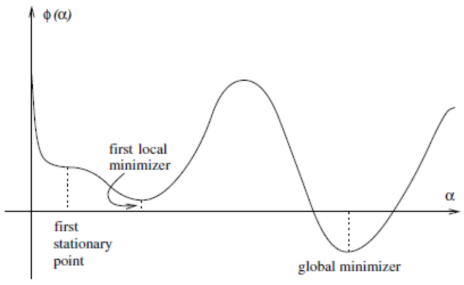
- η_k determines how far to move along the direction d_k.
- Fundamental trade-off:
  - **Too small: slow convergence**.
  - **Too large: divergence (overshooting)**.
- Viewed as a 1-D minimization problem:

$$\phi(\eta) \;=\; f(x_k + \eta\, d_k).$$

- Ideally: find the global minimum of $\phi(\eta)$.
- In practice: exact global minimization is computationally expensive, especially in high dimensions.

Inexact Line Search

- Practical algorithms rarely compute the exact minimizer.
- Instead, perform an **inexact line search**:
  - Choose η_k that is "sufficiently good" by precise criteria.
- The most widely used criteria: **Wolfe conditions** and their stronger variant.



Wolfe Conditions

The **Wolfe conditions** use two inequalities on η_k to:

- Ensure **sufficient decrease**.
- Avoid steps that are **too small**.

Condition (i): Sufficient Decrease (Armijo Condition)

$$f(x_k + \eta_k d_k) \;\leq\; f(x_k) + c_1\, \eta_k\, \nabla f_k^{\mathsf{T}} d_k.$$

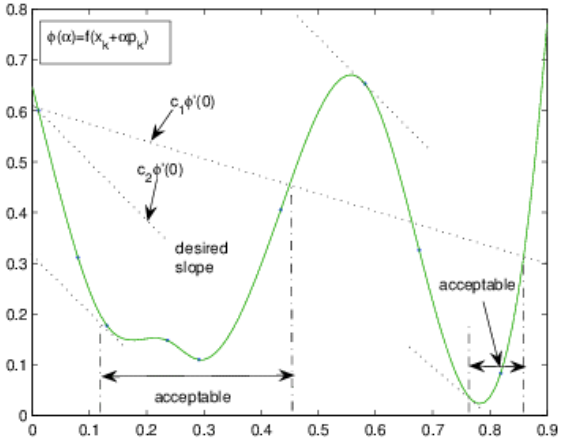
- Choose $c_1 \in (0, 1)$, typically very small (e.g., 10^{-4}).
- Forces the actual decrease in f to be a fixed fraction of the linear prediction.
- In 1-D form: $\phi(\eta_k) \leq \phi(0) + c_1\, \eta_k\, \phi'(0)$.

Condition (ii): Curvature Condition

$$\nabla f(x_k + \eta_k d_k)^{\mathsf{T}} d_k \;\geq\; c_2 \nabla f_k^{\mathsf{T}} d_k.$$

- Select $c_2 \in (c_1, 1)$, often close to 0.9.
- Prevents overly small steps: requires the directional slope at the new point to have flattened relative to the initial slope.
- In scalar form: $\phi'(\eta_k) \geq c_2 \phi'(0)$.

Geometric Understanding of Wolfe Conditions



Acceptable η_k lies where $\phi(\eta)$ is below the Armijo line *and* the slope is no steeper than $c_2\phi'(0)$.

Strong Wolfe Conditions

The **Strong Wolfe conditions** tighten the curvature requirement:

Strong Wolfe Conditions

$$f(x_k + \eta_k d_k) \leq f(x_k) + c_1 \eta_k \nabla f_k^\mathsf{T} d_k,$$
$$\left|\nabla f(x_k + \eta_k d_k)^\mathsf{T} d_k\right| \leq c_2 \left|\nabla f_k^\mathsf{T} d_k\right|.$$

For $0 < c_1 < c_2 < 1$, such an interval of step sizes **always exists**.

Rate of Convergence

Optimization methods pursue two distinct objectives:

- **Global Convergence**
  - From an arbitrary initial point, the sequence converges to a **critical point** (where $\nabla f = 0$).
  - Does not guarantee convergence to a local or global minimizer - saddle points and local maxima also satisfy $\nabla f = 0$.
  - Emphasis: **robustness**.
- **Fast Rate of Convergence**
  - Near x^*, the sequence closes the remaining gap rapidly.
  - Emphasis: **speed**.

We now classify convergence rates accordingly.

Q-Linear Convergence

Definition

Suppose $x_k \to x^*$. If there exist $c \in [0, 1)$ and $N \geq 0$ such that

$$\|x_{k+1} - x^*\| \leq c \|x_k - x^*\|, \qquad \forall k \geq N,$$

- Each step reduces the error by a fixed ratio.
- The decay is geometric.
- Characterizes **linear convergence** — typically slow.

Q-Order Convergence

Definition

Assume there exist constants $C > 0$, $p > 1$, $N \geq 0$ such that

$$\|x_{k+1} - x^*\| \leq C \|x_k - x^*\|^p, \qquad \forall k \geq N.$$

- Error shrinks at a higher-order rate.
- $p = 2$: **Q-quadratic convergence**.
- $p = 3$: **Q-cubic convergence**.

Q-Superlinear Convergence

Definition

If there exists a sequence $\{c_k\}$ with $c_k \to 0$ such that

$$\|x_{k+1} - x^*\| \leq c_k \|x_k - x^*\|.$$

- Faster than linear, slower than quadratic.
- Characteristic of **Quasi-Newton methods**.

Note. "Q-" stands for "Quotient": the rate is defined by the ratio of successive errors.

Comparison of Convergence Properties

Algorithm	**Update Rule**	**Rate**	**Global**	**Local**
Steepest Descent	$x_{k+1} = x_k - \eta_k \nabla f_k$	Linear	Yes (w/ Wolfe)	—
Quasi-Newton	$x_{k+1} = x_k - \eta_k B_k^{-1} \nabla f_k$	Superlinear	Yes (w/ Wolfe)	—
Newton's Method	$x_{k+1} = x_k - H_k^{-1} \nabla f_k$	Quadratic	No	Yes

Coordinate Descent Methods

- Minimize f by updating **one coordinate at a time**.
- Keep all other variables fixed; optimize along a single axis.
- Given $x = (x_1, \ldots, x_n)$, cycle through $e_1, \ldots, e_n$.

Example ($n = 2$). Alternate between two 1-D problems:

$$x_1^{(k+1)} = \arg\min_{x_1} f\left(x_1,\, x_2^{(k)}\right), \qquad x_2^{(k+1)} = \arg\min_{x_2} f\left(x_1^{(k+1)},\, x_2\right).$$

General step. At iteration k, update only x_j:

$$x_j^{(k+1)} = \arg\min_{x_j} \; f\left(x_1^{(k+1)}, \ldots, x_{j-1}^{(k+1)},\, x_j,\, x_{j+1}^{(k)}, \ldots, x_n^{(k)}\right).$$

Coordinate Descent: Properties

Advantages

- Reduces each step to a 1-D problem.
- Can work without gradients.
- Performs well when variables interact weakly.

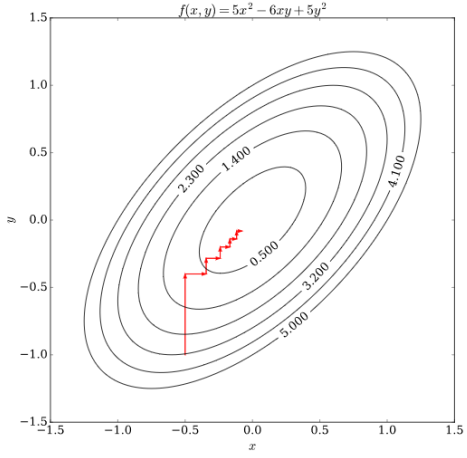
Disadvantages

- Slow convergence when variables are strongly correlated.
- Sweeping all n coordinates can be expensive for large n.

Alternative Names

- **Alternating Variables Method:** switch variables one by one.
- **Block Coordinate Descent:** update several coordinates together.

Coordinate Descent: Visual Example



Quasi-Newton Methods

Quasi-Newton Methods

Idea

- Build and update an **approximation of the Hessian**; avoid computing second derivatives directly.
- At each iteration, evaluate **only the gradient**.
- Use the change in successive gradients to update the Hessian estimate (or its inverse).

Position among methods

- Bridges Newton's rapid convergence and gradient descent's low cost.
- Fast local convergence (typically **superlinear**) without forming or storing the exact Hessian.
- Especially effective in high dimensions, or when the true Hessian is impractical.

The BFGS Method

- Most widely used algorithm in the Quasi-Newton family.
- Named after its originators: **Broyden, Fletcher, Goldfarb, Shanno**.

Core idea

- Iteratively approximate the Hessian (or its inverse).
- Use **only gradient** information.
- No explicit calculation of second derivatives.

BFGS: Problem Setup and Quadratic Model

Unconstrained problem

$$\min_{x \in \mathbb{R}^n} f(x), \quad f \in C^1 \text{ (usually } C^2).$$

Quadratic model at x_k

$$q_k(d) \;=\; f_k + \nabla f_k^{\mathsf{T}} d + \tfrac{1}{2}\, d^{\mathsf{T}} B_k\, d.$$

- $f_k = f(x_k)$, $\nabla f_k = \nabla f(x_k)$.
- $B_k \in \mathbb{R}^{n \times n}$: Hessian approximation; **symmetric and positive definite**.
- Matching conditions: $q_k(0) = f_k$, $\nabla q_k(0) = \nabla f_k$.

BFGS: Search Direction and Line Search

Direction minimizing q_k

$$d_k \;=\; -B_k^{-1}\nabla f_k$$

Line search update

$$x_{k+1} = x_k + \eta_k\, d_k.$$

- $\eta_k > 0$ chosen to satisfy the **Wolfe** conditions.

BFGS: Secant Condition

Define the quasi-Newton vectors

$$s_k \;=\; x_{k+1} - x_k, \qquad y_k \;=\; \nabla f_{k+1} - \nabla f_k.$$

The new Hessian approximation B_{k+1} should be consistent with the step just taken and the observed gradient change:

$$B_{k+1}\, s_k \;=\; y_k. \quad \textbf{(secant equation)}$$

- Mimics the property $\nabla^2 f(x)\,\Delta x \approx \Delta(\nabla f)$ of the true Hessian.

BFGS: Curvature Condition

For B_{k+1} to remain positive definite, we need the **curvature condition**:

$$s_k^T y_k > 0.$$

- **A Wolfe line search automatically guarantees this.**
- No extra check or safeguard is needed in the algorithm.

BFGS Inverse Hessian Update

- Many matrices B_{k+1} satisfy the secant equation.
- BFGS picks one specific update that is theoretically well-grounded and works well in practice.

BFGS inverse update formula

$$H_{k+1} = \left(I - \rho_k s_k y_k^\mathsf{T}\right) H_k \left(I - \rho_k y_k s_k^\mathsf{T}\right) + \rho_k s_k s_k^\mathsf{T}, \qquad \rho_k = \frac{1}{y_k^\mathsf{T} s_k}.$$

The BFGS Algorithm

Algorithm

- **Input:** starting point x_0, tolerance $\varepsilon > 0$, initial inverse Hessian H_0; set $k \leftarrow 0$.
- **while** $\|\nabla f_k\| > \varepsilon$:
  1. Search direction: $d_k = -H_k \nabla f_k$.
  2. Line search: choose η_k satisfying the Wolfe conditions; set $x_{k+1} = x_k + \eta_k d_k$.
  3. Define $s_k = x_{k+1} - x_k$ and $y_k = \nabla f_{k+1} - \nabla f_k$.
  4. Update $H_{k+1} = \left(I - \rho_k s_k y_k^{\mathsf{T}}\right) H_k \left(I - \rho_k y_k s_k^{\mathsf{T}}\right) + \rho_k s_k s_k^{\mathsf{T}}$.
  5. $k \leftarrow k + 1$.
- **end while**

BFGS: Computational Cost and Practicality

Storage and per-iteration cost

- Storing B_k or H_k: $O(n^2)$.
- Search direction $d_k = -H_k \nabla f_k$: $O(n^2)$.
- BFGS rank-two update of H_k: $O(n^2)$.

Practical limitations

- For n in the thousands+, $O(n^2)$ becomes prohibitive.
- For very large problems, the memory-efficient variant **L-BFGS** keeps only the last m pairs (s_i, y_i), reducing storage to $O(mn)$ (used in SciPy, PyTorch, ...).